

Project: Algoritmi genetici e progettazione di reti

minimum spanning tree, vincoli di capacita' e ottimizzazione evolutiva

1. Obiettivi della dispensa

Questa dispensa introduce gli algoritmi genetici come caso di studio per un corso di metodi computazionali per modelli stocastici, applicati al problema della progettazione di una rete di distribuzione con vincoli di capacita'.

Gli obiettivi sono sette:

1. formalizzare il problema del minimum spanning tree e mostrare perche' l'algoritmo greedy di Kruskal e' ottimale in assenza di vincoli;
2. introdurre i vincoli di capacita' e mostrare perche' rompono la struttura che rende il greedy ottimale;
3. presentare gli algoritmi genetici come framework generale di ottimizzazione stocastica basato su popolazione;
4. discutere la rappresentazione cromosomica, gli operatori genetici (selezione, crossover, mutazione) e la funzione di fitness;
5. affrontare il ruolo della popolazione iniziale e il trade-off tra qualita' iniziale e diversita' genetica;
6. applicare il GA al problema di rete su dati reali dei capoluoghi di regione italiani;
7. discutere il confronto con il simulated annealing e la variante degli schedule.

Dal punto di vista del corso, questo progetto completa il quadro dell'ottimizzazione stocastica: dove SA lavora su una soluzione singola con una dinamica di tipo MCMC, il GA lavora su una popolazione di soluzioni con una dinamica ispirata all'evoluzione biologica. Il confronto tra i due approcci e' uno dei messaggi metodologici piu' importanti del progetto.

2. Motivazione: reti di distribuzione e il costo dell'ottimalita' locale

2.1 Il problema concreto

Una regione deve costruire una rete di distribuzione — gas naturale, acqua potabile, fibra ottica, energia elettrica — che colleghi tutti i capoluoghi di provincia o di regione. Esistono molte tratte possibili da costruire, ciascuna con un costo proporzionale alla lunghezza e alla difficulta' del terreno. Ogni comune ha una domanda di capacita': quanta risorsa deve poter ricevere ogni giorno.

Il problema e' trovare il sottoinsieme di tratte da costruire che:

- connette tutti i comuni (nessun comune isolato);
- soddisfa la domanda di ogni comune (nessun collo di bottiglia);
- minimizza il costo totale di costruzione.

Esempio concreto. L'ANAS deve pianificare il potenziamento della rete idrica in una regione del Sud. Ci sono 20 capoluoghi da collegare. Costruire una condotta tra due capoluoghi costa proporzionalmente alla distanza stradale tra di essi. Ogni capoluogo ha una domanda di portata d'acqua proporzionale alla sua popolazione. Una condotta ha una portata massima fissa. Come si progetta la rete al minimo costo?

2.2 Perche' la complessita' varia con i vincoli

Senza il vincolo di capacita', questo e' il problema del **minimum spanning tree** (MST): trovare l'albero di costo minimo che connette tutti i nodi. Ha soluzione esatta in tempo polinomiale.

Con il vincolo di capacita', il problema diventa molto piu' difficile. Non e' piu' sufficiente trovare l'albero piu' corto: bisogna garantire che ogni tratta possa sostenere il flusso richiesto dai nodi a valle. Questo puo' richiedere di costruire tratte piu' costose, aggiungere tratte ridondanti, o scegliere percorsi alternativi. Il problema e' NP-hard nella sua forma generale.

Questa progressione — da problema facile a problema difficile aggiungendo un vincolo naturale — e' il cuore didattico del progetto.

3. Il minimum spanning tree e l'algoritmo di Kruskal

3.1 Definizione

Dato un grafo non orientato $G = (V, E)$ con n nodi e m archi, dove ogni arco (i, j) ha un costo $c_{ij} \geq 0$, un **albero di copertura** (spanning tree) e' un sottoinsieme di archi $T \subseteq E$ tale che:

- T connette tutti i nodi (il grafo (V, T) e' connesso);
- T non contiene cicli (e' un albero);
- $|T| = n - 1$ (un albero su n nodi ha esattamente $n - 1$ archi).

Il **minimum spanning tree** e' l'albero di copertura con costo totale minimo:

$$T^* = \arg \min_{T \text{ spanning tree}} \sum_{(i,j) \in T} c_{ij}.$$

Esempio concreto. I nodi sono i 20 capoluoghi di regione italiani. Il costo di ogni arco e' la distanza in km tra i due capoluoghi (calcolata con la formula haversine). Il MST e' la rete di condotte piu' corta che connette tutti i capoluoghi.

3.2 L'algoritmo di Kruskal

L'algoritmo di Kruskal e' un algoritmo greedy che costruisce il MST in modo incrementale:

1. ordina tutti gli archi per costo crescente;
2. inizializza $T = \emptyset$ e una struttura Union-Find che tiene traccia delle componenti connesse;
3. per ogni arco (i, j) in ordine di costo crescente:
 - se i e j appartengono a componenti connesse diverse: aggiungi (i, j) a T e unisci le due componenti;
 - altrimenti: scarta l'arco (aggiungerlo creerebbe un ciclo).
4. restituisci T .

La complessita' e' $O(m \log m)$, dominata dall'ordinamento degli archi.

3.3 Perche' il greedy e' ottimale: la struttura matroidale

Il motivo per cui Kruskal garantisce l'ottimo globale — non un minimo locale — e' che il problema del MST ha una struttura algebrica molto speciale chiamata **matroide**.

Intuitivamente, un matroide e' una struttura in cui la proprieta' di “essere un sottoinsieme indipendente” (in questo caso: un sottoinsieme aciclico di archi) soddisfa una proprieta' di scambio: se ho due insiemi indipendenti di dimensioni diverse, posso sempre aggiungere un elemento del piu' grande al piu' piccolo mantenendo l'indipendenza.

Questa proprieta' garantisce che l'algoritmo greedy — aggiungi sempre l'arco di costo minimo che non crea cicli — non puo' mai “perdere” la soluzione ottimale per una scelta locale sbagliata. Il greedy e' ottimale globalmente.

Questo e' il punto chiave. L'ottimalita' del greedy non e' una coincidenza: e' la conseguenza diretta della struttura matroidale del problema. Quando si aggiungono vincoli di capacita', questa struttura si rompe, e con essa l'ottimalita' del greedy.

3.4 Il MST come benchmark

Il MST ha due ruoli nel progetto:

- come **soluzione di riferimento** per il problema senza vincoli: e' la rete di costo minimo quando la capacita' non e' un problema;
- come **ingrediente della popolazione iniziale** del GA: il MST e' un ottimo punto di partenza perche' ha costo basso, anche se potrebbe violare i vincoli di capacita'.

3.5 Dati: capoluoghi di regione italiani

Per mantenere il problema trattabile nei laboratori iniziali si usano i 20 capoluoghi di regione italiani. Il grafo e' completo: ci sono $\binom{20}{2} = 190$ archi possibili, uno per ogni coppia di capoluoghi. Il costo di ogni arco e' la distanza haversine in km.

La domanda di ogni nodo e' proporzionale alla popolazione del capoluogo. La capacita' di ogni tratta e' un parametro fisso del problema.

4. Quando il greedy non basta: vincoli di capacita'

4.1 Il modello con capacita'

Aggiungiamo al problema MST il seguente vincolo: ogni tratta ha una **capacita' massima** $C_{\max}$ — la portata massima di risorsa che puo' trasportare. Ogni nodo i ha una **domanda** $d_i > 0$.

In una rete albero con sorgente nel nodo radice, il flusso sulla tratta (i, j) e' uguale alla somma delle domande di tutti i nodi nel sottoalbero a valle di (i, j) . Se questo flusso supera $C_{\max}$, la tratta e' sovraccarica.

Una rete valida deve soddisfare:

$$\text{flusso su } (i, j) = \sum_{k \in \text{sottoalbero}(j)} d_k \leq C_{\max} \quad \forall (i, j) \in T.$$

4.2 Perche' il MST puo' violare i vincoli

Il MST minimizza il costo totale senza tenere conto dei flussi. In una regione con popolazione molto concentrata, il MST potrebbe instradare tutto il flusso attraverso poche tratte centrali che diventano colli di bottiglia.

Esempio concreto. Nel Nord Italia, il MST tende a instradare il flusso da Torino, Milano, Venezia, Bologna attraverso poche tratte nell'area padana. Con una capacita' limitata, queste tratte si saturano. Per soddisfare i vincoli bisogna aggiungere tratte alternative — a costo maggiore — che bypassano i colli di bottiglia.

Il MST “rattoppato” (aggiungi la tratta piu’ economica che risolve ogni violazione) e’ una soluzione valida ma subottimale: la scelta locale di quale tratta aggiungere puo’ creare nuove violazioni o essere molto piu’ costosa della soluzione ottimale globale.

4.3 La necessita’ di un approccio globale

Con vincoli di capacita’, non esiste un algoritmo greedy che garantisca l’ottimo. Ogni decisione locale — quale tratta aggiungere — dipende dall’intera struttura della rete e dalla distribuzione dei flussi. Serve un metodo che esplori lo spazio delle soluzioni in modo globale.

E’ qui che entrano gli algoritmi genetici.

5. Gli algoritmi genetici

5.1 Idea generale

Gli algoritmi genetici (GA) sono una famiglia di metodi di ottimizzazione ispirati all’evoluzione biologica. Lavorano su una **popolazione** di soluzioni candidate che evolvono attraverso generazioni successive.

A ogni generazione:

1. le soluzioni vengono valutate tramite una **funzione di fitness**;
2. le soluzioni migliori vengono selezionate per la **riproduzione**;
3. le soluzioni selezionate si combinano tramite **crossover** per produrre nuovi individui;
4. i nuovi individui vengono perturbati casualmente tramite **mutazione**;
5. la nuova generazione sostituisce (parzialmente o totalmente) la precedente.

Dopo molte generazioni, la popolazione converge verso soluzioni di alta qualita’.

5.2 Analogia biologica e differenza con SA

L’analogia con l’evoluzione biologica e’ esplicita: i cromosomi codificano soluzioni, la fitness misura la qualita’, la selezione favorisce i migliori, il crossover ricombina informazione tra individui diversi, la mutazione introduce variabilita’ casuale.

La differenza fondamentale con il simulated annealing e’ che SA lavora su una **soluzione singola** che percorre lo spazio delle soluzioni, mentre GA lavora su una **popolazione** di soluzioni che evolvono parallelamente. Il GA esplora piu’ regioni dello spazio simultaneamente, il che lo rende meno sensibile ai minimi locali ma piu’ costoso per valutazione.

Analogia con la replicator equation. La selezione proporzionale alla fitness nel GA e’ formalmente analoga alla replicator equation: le soluzioni con fitness maggiore della media crescono di frequenza nella popolazione, quelle con fitness minore decrescono. GA e dinamiche replicative descrivono lo stesso meccanismo

di selezione — uno in un contesto computazionale, l'altro in un contesto evolutivo o sociale.

5.3 La rappresentazione cromosomica

Per il problema di network design, ogni **cromosoma** e' un vettore binario di lunghezza m :

$$\mathbf{x} = (x_1, x_2, \dots, x_m) \in \{0, 1\}^m,$$

dove $x_e = 1$ se la tratta e e' inclusa nella rete e $x_e = 0$ altrimenti.

Questa rappresentazione e' molto naturale: ogni bit corrisponde alla decisione di costruire o non costruire una tratta. Il numero di cromosomi possibili e' 2^m , ma solo quelli che corrispondono a reti connesse sono soluzioni valide.

Esempio. Con 20 nodi e grafo completo, $m = 190$ e il numero di cromosomi possibili e' $2^{190} \approx 10^{57}$. Solo una piccola frazione corrisponde a reti connesse che soddisfano i vincoli di capacita'.

5.4 La funzione di fitness

La funzione di fitness deve:

- assegnare un valore alto alle soluzioni di basso costo che soddisfano tutti i vincoli;
- penalizzare le soluzioni che violano i vincoli (rete non connessa, capacita' insufficiente);
- essere calcolabile in tempo ragionevole.

Una forma comune e' la **penalita' additiva**:

$$f(\mathbf{x}) = -C(\mathbf{x}) - \lambda_1 \cdot \text{violazioni_connettivita}'(\mathbf{x}) - \lambda_2 \cdot \text{violazioni_capacita}'(\mathbf{x}),$$

dove $C(\mathbf{x})$ e' il costo totale della rete, λ_1 e λ_2 sono parametri di penalita', e le violazioni si misurano come numero di nodi disconnessi o eccesso di flusso sulle tratte sature.

Si massimizza f (o equivalentemente si minimizza $-f$).

La scelta dei parametri di penalita' e' delicata: penalita' troppo basse lasciano prevalere soluzioni invalide, penalita' troppo alte rendono il paesaggio di fitness dominato dai vincoli e oscurano le differenze di costo.

5.5 Selezione

La selezione determina quali individui si riproducono. Il metodo piu' comune e' la **selezione a torneo**: si estraggono casualmente k individui dalla popolazione, e il migliore (quello con fitness massima) viene selezionato per la riproduzione. Il processo si ripete per ottenere tutti i genitori necessari.

La selezione a torneo e' robusta, semplice da implementare, e permette di controllare la pressione selettiva tramite la dimensione k del torneo: con k grande, quasi sempre vince il migliore e la pressione e' alta; con $k = 2$ la pressione e' moderata.

Alternativa: selezione proporzionale alla fitness (roulette wheel). La probabilita' di selezione e' proporzionale alla fitness. E' il metodo piu' direttamente analogo alla replicator equation, ma e' piu' sensibile alle differenze di scala della fitness.

5.6 Crossover

Il crossover ricombina il materiale genetico di due genitori per produrre uno o due figli.

Crossover a un punto. Si sceglie casualmente un punto di taglio $k \in \{1, \dots, m-1\}$. Il primo figlio prende i bit $1, \dots, k$ dal primo genitore e i bit $k+1, \dots, m$ dal secondo. Il secondo figlio prende il complemento.

genitore 1: $[x_1, \dots, x_k \mid x_{k+1}, \dots, x_m]$

genitore 2: $[y_1, \dots, y_k \mid y_{k+1}, \dots, y_m]$

figlio 1: $[x_1, \dots, x_k \mid y_{k+1}, \dots, y_m]$

Crossover uniforme. Ogni bit del figlio viene preso dal primo o dal secondo genitore con probabilita' $1/2$. Produce maggiore diversita' ma distrugge piu' facilmente la struttura della soluzione.

Per il problema di network design, il crossover a un punto e' preferibile perche' tende a preservare blocchi contigui di tratte, che spesso corrispondono a sottoreti geograficamente coerenti.

5.7 Mutazione

La mutazione introduce perturbazioni casuali nei cromosomi figli. Per la rappresentazione binaria, ogni bit viene invertito con probabilita' p_m :

$$x_e \leftarrow 1 - x_e \quad \text{con probabilita' } p_m.$$

La probabilit  di mutazione p_m   tipicamente molto bassa: $p_m \in [1/m, 5/m]$, dove m   la lunghezza del cromosoma. Questo garantisce che in media uno o pochi bit vengano mutati per individuo.

La mutazione serve a:

- reintrodurre variabilit  genetica quando la popolazione converge;
- permettere di raggiungere regioni dello spazio non accessibili tramite crossover.

  l'analogo della temperatura residua nel SA: anche quando il GA   quasi convergito, la mutazione mantiene una piccola probabilit  di esplorare soluzioni nuove.

6. Popolazione iniziale e diversit  genetica

6.1 Il trade-off tra qualit  e diversit 

La scelta della popolazione iniziale   uno dei parametri pi  importanti del GA. C'  un trade-off fondamentale:

- **Popolazione di alta qualit **: iniziare vicino a buone soluzioni accelera la convergenza, ma riduce la diversit  e pu  intrappolare il GA in un intorno ristretto dello spazio.
- **Popolazione casuale**: massimizza la diversit  e la copertura dello spazio, ma parte lontano dalle soluzioni buone e pu  richiedere molte generazioni per convergere.

Questo trade-off   formalmente analogo al parametro di temperatura iniziale in SA:

- temperatura alta in SA $\approx$ popolazione casuale in GA: molta esplorazione, poco sfruttamento;
- temperatura bassa in SA $\approx$ popolazione concentrata vicino al MST: molto sfruttamento, poca esplorazione.

6.2 Popolazione mista: la strategia consigliata

La strategia pi  robusta   una **popolazione mista**:

- una frazione $p_{MST} \in [0.2, 0.3]$ degli individui   costruita a partire dal MST con piccole perturbazioni casuali (si aggiungono o rimuovono alcune tratte casuali, riparando la connettivit  se necessario);
- la restante frazione   generata casualmente, garantendo solo la connettivit  di base.

Questa strategia combina i vantaggi:

- gli individui basati sul MST portano informazione sul costo basso e accelerano la convergenza;

- gli individui casuali garantiscono diversita' e permettono al GA di esplorare regioni lontane dal MST dove potrebbero stare le soluzioni ottimali con vincoli di capacita'.

6.3 Diversita' genetica come misura di salute della popolazione

La **diversita' genetica** della popolazione si puo' misurare come la distanza di Hamming media tra tutti i pari di individui:

$$D = \frac{2}{P(P-1)} \sum_{1 \leq a < b \leq P} d_H(\mathbf{x}^{(a)}, \mathbf{x}^{(b)}),$$

dove P e' la dimensione della popolazione e d_H e' la distanza di Hamming (numero di bit diversi).

Se D crolla verso zero, la popolazione e' convergita — tutti gli individui sono quasi identici. In questo caso si puo' iniettare nuovi individui casuali (**restart parziale**) per riportare diversita'.

Monitorare D nel corso delle generazioni e' analogo a monitorare la temperatura in SA: entrambi misurano quanto il sistema sta esplorando lo spazio.

7. Riparazione delle soluzioni non valide

7.1 Il problema della validita'

Non tutti i cromosomi corrispondono a reti valide. Un cromosoma con pochi bit a 1 potrebbe produrre una rete non connessa; uno con molti bit a 1 potrebbe essere connesso ma non soddisfare i vincoli di capacita' in alcuni nodi.

Ci sono due approcci per gestire le soluzioni non valide:

Approccio 1: penalita'. Le soluzioni invalide vengono comunque valutate e mantenute nella popolazione, ma la loro fitness e' penalizzata in proporzione alle violazioni. E' semplice da implementare ma richiede di tarare i parametri di penalita'.

Approccio 2: riparazione. Dopo ogni operazione genetica (crossover e mutazione), le soluzioni invalide vengono "riparate" con un algoritmo deterministico che le rende valide. E' piu' costoso ma garantisce che tutta la popolazione sia sempre formata da soluzioni valide.

7.2 Algoritmo di riparazione per la connettivita'

Una soluzione non connessa puo' essere riparata in modo greedy:

1. identifica le componenti connesse della rete corrente;

2. finche' ci sono piu' di una componente: aggiungi l'arco di costo minimo che connette due componenti diverse (come in Kruskal, ma partendo dalla rete corrente invece che dal grafo vuoto);
3. restituisci la rete riparata.

Questa riparazione e' veloce e produce una soluzione connessa di buona qualita'.

7.3 Algoritmo di riparazione per i vincoli di capacita'

Una rete connessa che viola i vincoli di capacita' su alcune tratte puo' essere riparata:

1. identifica tutte le tratte sature (flusso $> C_{\max}$);
2. per ogni tratta satura (i, j) : aggiungi la tratta alternativa di costo minimo che riduce il flusso su (i, j) ridistribuendo parte del flusso su un percorso alternativo;
3. ripeti finche' tutti i vincoli sono soddisfatti.

Questa riparazione e' piu' complessa ma garantisce la validita' della soluzione.

Per semplicita', nel laboratorio si puo' usare l'approccio a penalita' per i vincoli di capacita' e la riparazione solo per la connettivita'.

8. Il GA per network design: schema completo

8.1 Parametri principali

- P : dimensione della popolazione (tipicamente 50-200);
- G : numero di generazioni (tipicamente 100-500);
- p_c : probabilita' di crossover (tipicamente 0.7-0.9);
- p_m : probabilita' di mutazione per bit (tipicamente $1/m - 5/m$);
- k_{torneo} : dimensione del torneo (tipicamente 2-5);
- p_{MST} : frazione della popolazione iniziale basata sul MST (0.2-0.3);
- λ_1, λ_2 : parametri di penalita' per violazioni di connettivita' e capacita'.

8.2 Schema dell'algoritmo

1. **Inizializzazione**: genera la popolazione iniziale mista (MST perturbato + casuale);
2. **Valutazione**: calcola la fitness di ogni individuo;
3. **Evoluzione** (ripeti per G generazioni):
 - **Selezione**: seleziona i genitori tramite torneo;
 - **Crossover**: genera i figli con probabilita' p_c ;
 - **Mutazione**: perturba i figli con probabilita' p_m per bit;
 - **Riparazione**: correggi le soluzioni non connesse;
 - **Valutazione**: calcola la fitness dei nuovi individui;
 - **Sostituzione**: forma la nuova generazione (elitismo + nuovi individui);

- **Monitoraggio:** registra fitness migliore, media, diversita' genetica;
4. **Risultato:** restituisci il miglior individuo trovato.

8.3 Elitismo

L'**elitismo** consiste nel preservare i migliori e individui della generazione corrente nella generazione successiva, senza sottoporli a crossover o mutazione. Garantisce che la qualita' della soluzione migliore non diminuisca mai tra una generazione e la successiva.

Con elitismo $e = 1$ (si preserva solo il miglior individuo), il GA converge in modo monotono: la curva della fitness migliore e' non decrescente. Senza elitismo, la fitness puo' fluttuare.

9. Connessione con il simulated annealing

9.1 Confronto strutturale

Caratteristica	Simulated Annealing	Algoritmo Genetico
Struttura	soluzione singola	popolazione
Esplorazione	temperatura decrescente	mutazione + crossover
Fuga dai minimi locali	accettazione probabilistica	diversita' della popolazione
Convergenza	garantita (schedule log.)	non garantita in generale
Costo per iterazione	basso	alto (valuta P soluzioni)
Parallelizzabilita'	bassa	alta

9.2 Quando preferire GA a SA

GA e' preferibile a SA quando:

- il problema ha struttura modulare: le soluzioni buone si costruiscono combinando "blocchi" di bit buoni, e il crossover sfrutta questa struttura;
- il costo computazionale per valutazione e' basso e si possono valutare molte soluzioni in parallelo;
- si vuole esplorare contemporaneamente piu' regioni dello spazio, ad esempio per identificare piu' soluzioni "buone" invece del solo ottimo.

SA e' preferibile quando:

- il problema ha struttura continua o quando le mosse locali sono ben definite;
- il budget computazionale e' limitato e si preferisce un'esplorazione profonda di una regione dello spazio;
- la teoria di convergenza e' importante (SA ha garanzie teoriche piu' solide).

Per il problema di network design, entrambi funzionano bene. GA e' piu' naturale perche' la struttura a grafo del cromosoma si presta al crossover: due reti valide

possono essere combinate per ottenere una nuova rete valida ereditando sottoreti da entrambe.

10. Variante: ottimizzazione di schedule

10.1 La stessa logica, cromosomi diversi

La stessa struttura di GA si applica a una classe molto diversa di problemi: l'ottimizzazione di orari e turni.

Esempi concreti:

- **Orario universitario:** assegnare corsi a aule e fasce orarie minimizzando conflitti (stesso studente in due corsi, stesso docente in due aule, aula troppo piccola per il corso);
- **Turni ospedalieri:** assegnare infermieri ai turni rispettando vincoli su ore massime, specializzazioni richieste, preferenze individuali;
- **Pianificazione di interventi di manutenzione:** schedulare ispezioni di infrastrutture su un arco temporale minimizzando i tempi di fermo.

10.2 Rappresentazione cromosomica per schedule

Per un orario con n corsi e m slot (combinazioni aula-orario), il cromosoma e' una **permutazione** o una **matrice di assegnazione** invece di un vettore binario:

$$\mathbf{x} = (x_1, x_2, \dots, x_n), \quad x_i \in \{1, \dots, m\},$$

dove x_i e' lo slot assegnato al corso i .

Il crossover per permutazioni richiede operatori specializzati (PMX, OX, CX) che preservano la proprieta' di permutazione. La mutazione consiste nello scambiare casualmente due elementi della permutazione.

10.3 Il messaggio metodologico

La variante degli schedule mostra che GA e' un **framework**, non un algoritmo per un problema specifico. Cambiando la rappresentazione cromosomica e gli operatori genetici, lo stesso schema si adatta a problemi strutturalmente molto diversi. La logica — popolazione, selezione, crossover, mutazione, fitness — rimane identica.

Questa flessibilita' e' il principale vantaggio di GA rispetto a SA: SA richiede di definire una “mossa locale” che dipende fortemente dalla struttura del problema, mentre GA richiede solo di scegliere una rappresentazione e operatori coerenti con essa.

11. Domande scientifiche che il modello permette di studiare

Il modello e' utile per affrontare domande precise.

1. Quanto e' subottimale il MST rattoppato rispetto alla soluzione GA per il problema con vincoli di capacita'?
2. Come dipende la qualita' della soluzione dal parametro di capacita' $C_{\max}$? Esiste una soglia critica sotto la quale il costo aumenta drasticamente?
3. Quale frazione di popolazione iniziale basata sul MST massimizza la qualita' della soluzione finale?
4. Come evolve la diversita' genetica nel corso delle generazioni? Quando crolla, la qualita' della soluzione smette di migliorare?
5. GA trova soluzioni migliori di SA per questo problema? In quanto tempo?
6. Come scala il tempo di calcolo del GA con la dimensione del problema (n nodi)?

12. Schema del laboratorio

12.1 Laboratorio 1 - MST e soluzione greedy

Obiettivo

Implementare Kruskal, calcolare il MST sui dati italiani, e verificare i vincoli di capacita'.

Attivita'

1. caricare i 20 capoluoghi di regione con coordinate e popolazione;
2. calcolare la matrice delle distanze haversine;
3. implementare Kruskal con Union-Find;
4. visualizzare il MST sulla mappa italiana;
5. fissare una capacita' $C_{\max}$ e verificare quante tratte del MST la violano.

Domande guida

- quali tratte del MST sono i colli di bottiglia?
- come cambia il numero di violazioni al variare di $C_{\max}$?
- la soluzione "MST rattoppato" e' molto piu' costosa del MST originale?

Output richiesto

- codice sorgente;
- visualizzazione del MST sulla mappa;
- tabella delle tratte violate per diversi valori di $C_{\max}$;
- costo del MST e del MST rattoppato a confronto.

12.2 Laboratorio 2 - Implementazione del GA

Obiettivo

Implementare il GA con popolazione mista e confrontare con il MST rattoppato.

Attività

1. implementare la rappresentazione cromosomica, la fitness con penalità, e gli operatori genetici;
2. implementare la riparazione della connettività;
3. eseguire il GA con $P = 100$, $G = 200$, $p_{\text{MST}} = 0.25$;
4. confrontare il costo della soluzione GA con il MST rattoppato.

Domande guida

- il GA trova soluzioni migliori del MST rattoppato?
- come evolve la fitness migliore e media nel corso delle generazioni?
- quante generazioni servono per convergere?

Output richiesto

- codice sorgente;
- curva della fitness migliore e media vs generazioni;
- visualizzazione della rete ottimale trovata dal GA;
- tabella comparativa MST rattoppato vs GA.

12.3 Laboratorio 3 - Ruolo della popolazione iniziale e diversità genetica

Obiettivo

Studiare come la composizione della popolazione iniziale influenza la qualità della soluzione e la diversità genetica.

Attività

1. confrontare tre popolazioni iniziali: 100% casuale, 25% MST, 100% MST perturbato;
2. per ognuna, eseguire 10 run indipendenti;
3. misurare la diversità genetica D nel corso delle generazioni;
4. confrontare la distribuzione dei costi finali.

Domande guida

- la popolazione 100% MST converge più velocemente ma a soluzioni peggiori?
- quando la diversità D crolla, la fitness smette di migliorare?
- quale composizione iniziale è il miglior compromesso?

Output richiesto

- boxplot dei costi finali per le tre inizializzazioni;
- curve della diversita' genetica vs generazioni;
- commento sul trade-off qualita'/diversita'.

12.4 Laboratorio 4 - Confronto con SA e analisi della capacita' critica

Obiettivo

Confrontare GA e SA sullo stesso problema e identificare la capacita' critica sotto la quale il costo esplode.

Attivita'

1. implementare SA per lo stesso problema di network design;
2. confrontare qualita' della soluzione e tempo di calcolo tra GA e SA;
3. variare $C_{\max}$ su una griglia di valori e misurare il costo ottimale per ognuno;
4. identificare la soglia critica di capacita'.

Domande guida

- GA o SA trova soluzioni migliori? In quanto tempo?
- esiste un valore critico di $C_{\max}$ sotto il quale il costo aumenta rapidamente?
- la soglia critica corrisponde alla capacita' necessaria per sostenere il flusso sul MST originale?

Output richiesto

- tabella comparativa GA vs SA (costo, tempo, varianza);
- grafico del costo ottimale vs $C_{\max}$;
- commento sulla transizione di fase nel costo.

13. Perche' questo e' un buon case study per il corso

Questo progetto e' particolarmente adatto a un corso di metodi computazionali per modelli stocastici per almeno cinque ragioni.

Primo, la progressione greedy $\rightarrow$ GA e' didatticamente molto pulita: si mostra prima che il greedy e' ottimale per una ragione precisa (struttura matroidale), poi che aggiungere un vincolo naturale rompe quella struttura, poi che GA risolve il problema piu' generale. Lo studente capisce non solo come funziona GA, ma quando e' necessario.

Secondo, la connessione con la replicator equation rende GA non solo un nuovo strumento ma un collegamento tematico con un modello gia' visto.

Terzo, la discussione sulla diversita' genetica come analogo della temperatura in SA unifica concettualmente i due algoritmi di ottimizzazione del corso.

Quarto, i dati sono reali: i capoluoghi di regione italiani con le loro coordinate e popolazioni danno un problema geograficamente concreto e visivamente immediato.

Quinto, la variante degli schedule mostra la flessibilita' del framework GA senza richiedere una seconda implementazione completa.

14. Conclusione

Gli algoritmi genetici mostrano che la stocasticita' non serve solo per simulare fenomeni o campionare distribuzioni: puo' essere il motore di un processo di ricerca evolutivo che esplora spazi di soluzioni troppo grandi per qualsiasi approccio esatto.

La progressione dal MST al network design con capacita' illustra un principio generale: i problemi difficili nascono quando si aggiungono vincoli che rompono la struttura algebrica che rende i problemi facili. Riconoscere questa struttura — e sapere quando manca — e' una competenza fondamentale dell'ottimizzazione computazionale.

Dal punto di vista metodologico, il progetto combina in modo naturale:

- algoritmo greedy esatto con garanzie teoriche (Kruskal);
- algoritmo evolutivo stocastico (GA);
- dati geografici reali;
- analisi della popolazione e della diversita' genetica;
- confronto con SA;
- variante per un problema di tipo diverso (schedule).

Il messaggio concettuale piu' importante e' che GA non e' un algoritmo ma un framework: cambiando la rappresentazione e gli operatori, la stessa logica evolutiva si adatta a problemi strutturalmente molto diversi.

15. Bibliografia minima

1. Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
2. Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
3. Kruskal, J. B. (1956). On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem. *Proceedings of the American Mathematical Society*, 7(1), 48-50.
4. Garey, M. R., and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman.

5. Deb, K. (2001). Multi-Objective Optimization using Evolutionary Algorithms. Wiley.
-

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python il GA per il problema di network design sui capoluoghi di regione italiani.

A.1 Librerie minime

```
import random
import math
import statistics
import matplotlib.pyplot as plt
```

A.2 Dati: capoluoghi di regione italiani

```
# Ogni voce: (nome, latitudine, longitudine, popolazione)
CAPOLUOGHI_REGIONE = [
    ('Torino',          45.4064, 11.8768, 847287),
    ('Aosta',           45.7372,  7.3197,  34062),
    ('Milano',           45.4642,  9.1900, 1371498),
    ('Trento',           46.0748, 11.1217, 120875),
    ('Venezia',          45.4408, 12.3155, 249961),
    ('Trieste',          45.6522, 13.7722, 200713),
    ('Genova',           44.4056,  8.9463, 565752),
    ('Bologna',          44.4939, 11.3428, 419663),
    ('Firenze',          43.7711, 11.2486, 367150),
    ('Ancona',           43.6158, 13.5189,  99470),
    ('Roma',             41.8933, 12.4828, 2751755),
    ('L Aquila',         42.3500, 13.3997,  69753),
    ('Campobasso',       41.5597, 14.6561,  48592),
    ('Napoli',           40.8358, 14.2488, 909048),
    ('Bari',             41.1253, 16.8667, 315606),
    ('Potenza',          40.6394, 15.8019,  66034),
    ('Catanzaro',        38.9097, 16.5878,  87639),
    ('Palermo',          38.1111, 13.3522, 636872),
    ('Cagliari',         39.2239,  9.1219, 154083),
    ('Perugia',          43.1119, 12.3886, 162621),
]
```

A.3 Distanza haversine e matrice delle distanze

```
def haversine(lat1, lon1, lat2, lon2):
    R = 6371.0
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlam = math.radians(lon2 - lon1)
    a = (math.sin(dphi / 2.0) ** 2
          + math.cos(phi1) * math.cos(phi2) * math.sin(dlam / 2.0) ** 2)
    return 2.0 * R * math.asin(math.sqrt(a))

def build_distance_matrix(cities):
    n = len(cities)
    dist = [[0.0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            d = haversine(cities[i][1], cities[i][2],
                          cities[j][1], cities[j][2])
            dist[i][j] = d
            dist[j][i] = d
    return dist

def build_edge_list(n, dist):
    edges = []
    for i in range(n):
        for j in range(i + 1, n):
            edges.append((dist[i][j], i, j))
    edges.sort()
    return edges
```

A.4 Union-Find per Kruskal

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
```

```

    if rx == ry:
        return False
    if self.rank[rx] < self.rank[ry]:
        rx, ry = ry, rx
    self.parent[ry] = rx
    if self.rank[rx] == self.rank[ry]:
        self.rank[rx] += 1
    return True

```

A.5 Algoritmo di Kruskal

```

def kruskal_mst(n, edges):
    uf = UnionFind(n)
    mst_edges = []
    mst_cost = 0.0

    for cost, i, j in edges:
        if uf.union(i, j):
            mst_edges.append((i, j))
            mst_cost += cost
            if len(mst_edges) == n - 1:
                break

    return mst_edges, mst_cost

def mst_to_chromosome(mst_edges, edges):
    edge_set = set()
    for i, j in mst_edges:
        edge_set.add((min(i, j), max(i, j)))

    chromosome = []
    for _, i, j in edges:
        key = (min(i, j), max(i, j))
        chromosome.append(1 if key in edge_set else 0)

    return chromosome

```

A.6 Funzione di costo e verifica dei vincoli

```

def chromosome_to_edge_set(chromosome, edges):
    return {(i, j) for bit, (_, i, j) in zip(chromosome, edges) if bit == 1}

def network_cost(chromosome, edges):
    return sum(cost for bit, (cost, i, j) in zip(chromosome, edges) if bit == 1)

```

```

def is_connected(chromosome, edges, n):
    active = chromosome_to_edge_set(chromosome, edges)
    if not active:
        return False

    adj = [[] for _ in range(n)]
    for i, j in active:
        adj[i].append(j)
        adj[j].append(i)

    visited = [False] * n
    stack = [0]
    visited[0] = True
    count = 1

    while stack:
        u = stack.pop()
        for v in adj[u]:
            if not visited[v]:
                visited[v] = True
                count += 1
                stack.append(v)

    return count == n

def capacity_violations(chromosome, edges, n, demands, c_max):
    active = chromosome_to_edge_set(chromosome, edges)

    adj = [[] for _ in range(n)]
    for i, j in active:
        adj[i].append(j)
        adj[j].append(i)

    # trova la radice (nodo con domanda massima, proxy per il nodo sorgente)
    root = demands.index(max(demands))

    # calcola il flusso su ogni arco con una visita DFS
    visited = [False] * n
    subtree_demand = [0.0] * n
    order = []

    stack = [(root, -1)]
    while stack:

```

```

    u, parent = stack.pop()
    if visited[u]:
        continue
    visited[u] = True
    order.append((u, parent))
    subtree_demand[u] = demands[u]
    for v in adj[u]:
        if not visited[v]:
            stack.append((v, u))

for u, parent in reversed(order):
    if parent >= 0:
        subtree_demand[parent] += subtree_demand[u]

violations = 0
excess = 0.0
for u, parent in order:
    if parent >= 0:
        flow = subtree_demand[u]
        if flow > c_max:
            violations += 1
            excess += flow - c_max

return violations, excess

```

A.7 Funzione di fitness

```

def fitness(chromosome, edges, n, demands, c_max,
            lambda1=10000.0, lambda2=1000.0):
    cost = network_cost(chromosome, edges)

    if not is_connected(chromosome, edges, n):
        disconnected = n - 1
        return -(cost + lambda1 * disconnected)

    viol_count, excess = capacity_violations(
        chromosome, edges, n, demands, c_max)

    return -(cost + lambda2 * excess)

```

La fitness e' negativa del costo piu' le penalita': si massimizza la fitness, equivalente a minimizzare il costo penalizzato.

A.8 Riparazione della connettività

```
def repair_connectivity(chromosome, edges, n):
    if is_connected(chromosome, edges, n):
        return chromosome[:]

    repaired = chromosome[:]

    # aggiungi archi in ordine di costo finche' il grafo e' connesso
    for idx, (cost, i, j) in enumerate(edges):
        if repaired[idx] == 0:
            repaired[idx] = 1
            if is_connected(repaired, edges, n):
                return repaired

    return repaired
```

A.9 Generazione della popolazione iniziale

```
def random_connected_chromosome(n, edges):
    # genera un cromosoma casuale e lo ripara
    m = len(edges)
    chrom = [random.randint(0, 1) for _ in range(m)]
    return repair_connectivity(chrom, edges, n)

def mst_perturbed_chromosome(mst_chromosome, edges, n, n_flips=5):
    m = len(edges)
    chrom = mst_chromosome[:]

    for _ in range(n_flips):
        idx = random.randint(0, m - 1)
        chrom[idx] = 1 - chrom[idx]

    return repair_connectivity(chrom, edges, n)

def initialize_population(pop_size, n, edges, mst_chromosome,
                          mst_fraction=0.25):
    population = []
    n_mst = int(pop_size * mst_fraction)

    for _ in range(n_mst):
        chrom = mst_perturbed_chromosome(mst_chromosome, edges, n)
        population.append(chrom)
```

```

for _ in range(pop_size - n_mst):
    chrom = random_connected_chromosome(n, edges)
    population.append(chrom)

return population

```

A.10 Operatori genetici

```

def tournament_selection(population, fitnesses, k=3):
    indices = random.sample(range(len(population)), k)
    best_idx = max(indices, key=lambda i: fitnesses[i])
    return population[best_idx][:]

```

```

def single_point_crossover(parent1, parent2):
    m = len(parent1)
    point = random.randint(1, m - 1)
    child1 = parent1[:point] + parent2[point:]
    child2 = parent2[:point] + parent1[point:]
    return child1, child2

```

```

def mutate(chromosome, p_mut):
    return [1 - bit if random.random() < p_mut else bit
            for bit in chromosome]

```

A.11 Misura della diversita' genetica

```

def hamming_distance(c1, c2):
    return sum(b1 != b2 for b1, b2 in zip(c1, c2))

```

```

def population_diversity(population, sample_size=50):
    n = len(population)
    if n < 2:
        return 0.0

    pairs = min(sample_size, n * (n - 1) // 2)
    total = 0.0
    count = 0

    for _ in range(pairs):
        i, j = random.sample(range(n), 2)
        total += hamming_distance(population[i], population[j])
        count += 1

```

```
return total / count if count > 0 else 0.0
```

A.12 Il loop principale del GA

```
def genetic_algorithm(n, edges, demands, c_max,
                     pop_size=100, n_generations=200,
                     p_crossover=0.8, p_mutation=None,
                     tournament_k=3, elitism=2,
                     mst_fraction=0.25,
                     lambda1=10000.0, lambda2=1000.0,
                     seed=None):
    if seed is not None:
        random.seed(seed)

    m = len(edges)
    if p_mutation is None:
        p_mutation = 2.0 / m

    # MST come punto di partenza
    mst_edges, mst_cost = kruskal_mst(n, edges)
    mst_chrom = mst_to_chromosome(mst_edges, edges)

    # popolazione iniziale
    population = initialize_population(
        pop_size, n, edges, mst_chrom, mst_fraction)

    def eval_fitness(chrom):
        return fitness(chrom, edges, n, demands, c_max, lambda1, lambda2)

    fitnesses = [eval_fitness(c) for c in population]

    best_chrom = max(range(pop_size), key=lambda i: fitnesses[i])
    best_fitness = fitnesses[best_chrom]
    best_solution = population[best_chrom][:]

    history_best = [best_fitness]
    history_mean = [statistics.mean(fitnesses)]
    history_diversity = [population_diversity(population)]

    for gen in range(n_generations):
        # elitismo: preserva i migliori
        elite_indices = sorted(range(pop_size),
                               key=lambda i: fitnesses[i],
                               reverse=True)[:elitism]
        new_population = [population[i][:] for i in elite_indices]
```



```

# genera il resto della nuova popolazione
while len(new_population) < pop_size:
    p1 = tournament_selection(population, fitnesses, tournament_k)
    p2 = tournament_selection(population, fitnesses, tournament_k)

    if random.random() < p_crossover:
        c1, c2 = single_point_crossover(p1, p2)
    else:
        c1, c2 = p1[:, :], p2[:, :]

    c1 = mutate(c1, p_mutation)
    c2 = mutate(c2, p_mutation)

    c1 = repair_connectivity(c1, edges, n)
    c2 = repair_connectivity(c2, edges, n)

    new_population.append(c1)
    if len(new_population) < pop_size:
        new_population.append(c2)

population = new_population
fitnesses = [eval_fitness(c) for c in population]

gen_best_idx = max(range(pop_size), key=lambda i: fitnesses[i])
gen_best_fitness = fitnesses[gen_best_idx]

if gen_best_fitness > best_fitness:
    best_fitness = gen_best_fitness
    best_solution = population[gen_best_idx][:]

history_best.append(best_fitness)
history_mean.append(statistics.mean(fitnesses))
history_diversity.append(population_diversity(population))

results = {
    "best_solution": best_solution,
    "best_cost": network_cost(best_solution, edges),
    "mst_cost": mst_cost,
    "history_best_fitness": history_best,
    "history_mean_fitness": history_mean,
    "history_diversity": history_diversity
}

return results

```

A.13 Grafici

```
def plot_ga_evolution(history_best, history_mean, history_diversity):
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8), sharex=True)

    ax1.plot([-f for f in history_best], label="miglior costo")
    ax1.plot([-f for f in history_mean], label="costo medio", alpha=0.6)
    ax1.set_ylabel("costo (km)")
    ax1.set_title("Evoluzione del GA: costo e diversita'")
    ax1.legend()

    ax2.plot(history_diversity, color="darkorange")
    ax2.set_xlabel("generazione")
    ax2.set_ylabel("diversita' genetica (Hamming medio)")

    plt.tight_layout()
    plt.show()

def plot_network(solution, edges, cities, title="Rete ottimale"):
    active = chromosome_to_edge_set(solution, edges)

    fig, ax = plt.subplots(figsize=(8, 10))

    for i, j in active:
        lons = [cities[i][2], cities[j][2]]
        lats = [cities[i][1], cities[j][1]]
        ax.plot(lons, lats, 'b-', linewidth=1.5, alpha=0.7)

    for idx, (name, lat, lon, pop) in enumerate(cities):
        size = 5 + pop / 100000
        ax.plot(lon, lat, 'o', color='steelblue',
                markersize=size, zorder=5)
        ax.annotate(name, (lon, lat), fontsize=7,
                    xytext=(3, 3), textcoords='offset points')

    ax.set_xlabel("longitudine")
    ax.set_ylabel("latitudine")
    ax.set_title(title)
    plt.tight_layout()
    plt.show()
```

A.14 Esempio completo

```
if __name__ == "__main__":
    cities = CAPOLUOGHI_REGIONE
```

```

n = len(cities)
demands = [c[3] / 1e5 for c in cities] # in unita' di 100k abitanti
c_max = 20.0 # capacita' massima per tratta

print(f"Calcolo matrice distanze ({n}x{n})...")
dist = build_distance_matrix(cities)
edges = build_edge_list(n, dist)
print(f"Numero di archi possibili: {len(edges)}")

# MST base
mst_edges, mst_cost = kruskal_mst(n, edges)
print(f"\nCosto MST: {mst_cost:.0f} km")

mst_chrom = mst_to_chromosome(mst_edges, edges)
viol, excess = capacity_violations(mst_chrom, edges, n, demands, c_max)
print(f"Violazioni capacita' nel MST: {viol} tratte, eccesso totale: {excess:.1f}")

# GA
print("\nEsecuzione GA...")
res = genetic_algorithm(
    n=n, edges=edges, demands=demands, c_max=c_max,
    pop_size=100, n_generations=300,
    p_crossover=0.8, tournament_k=3,
    elitism=2, mst_fraction=0.25,
    lambda1=50000.0, lambda2=5000.0,
    seed=42
)

print(f"Costo soluzione GA: {res['best_cost']:.0f} km")
print(f"Miglioramento rispetto al MST: "
      f"{(mst_cost - res['best_cost'])/mst_cost*100:.1f}%"
      if res['best_cost'] < mst_cost else
      f"Costo aggiuntivo per soddisfare vincoli: "
      f"{(res['best_cost'] - mst_cost)/mst_cost*100:.1f}%")

plot_ga_evolution(res["history_best_fitness"],
                  res["history_mean_fitness"],
                  res["history_diversity"])

plot_network(res["best_solution"], edges, cities,
             title="Rete ottimale GA (capoluoghi di regione)")
plot_network(mst_chrom, edges, cities,
             title="MST (senza vincoli di capacita')")

```

A.15 Conclusione dell'appendice

La struttura proposta rende molto trasparente la progressione $\text{MST} \rightarrow \text{GA}$. La funzione `kruskal_mst` mostra che il greedy e' sufficiente senza vincoli; la funzione `capacity_violations` mostra dove il MST non basta; il loop del GA mostra come la popolazione evolva verso soluzioni che soddisfano i vincoli.

La separazione tra `repair_connectivity` (vincolo strutturale, sempre applicato) e la penalita' nella fitness per i vincoli di capacita' riflette la scelta progettuale discussa nella teoria: riparare la connettivita' e' economico e garantisce che ogni individuo sia almeno una rete, mentre gestire i vincoli di capacita' con penalita' e' piu' flessibile.